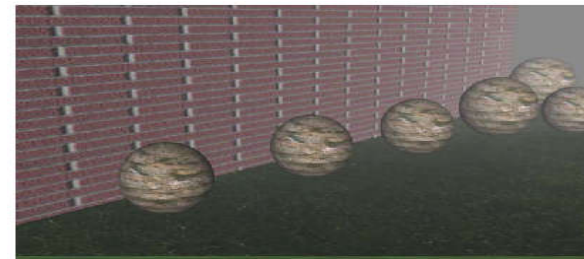
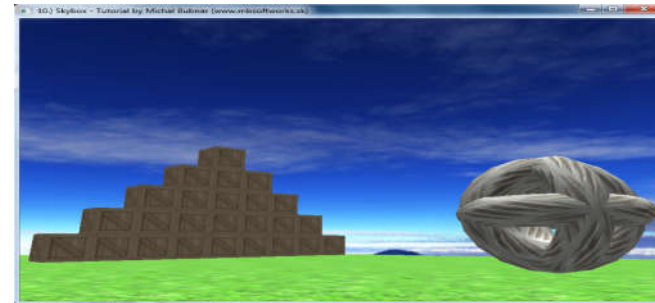
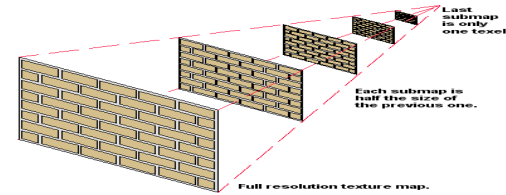


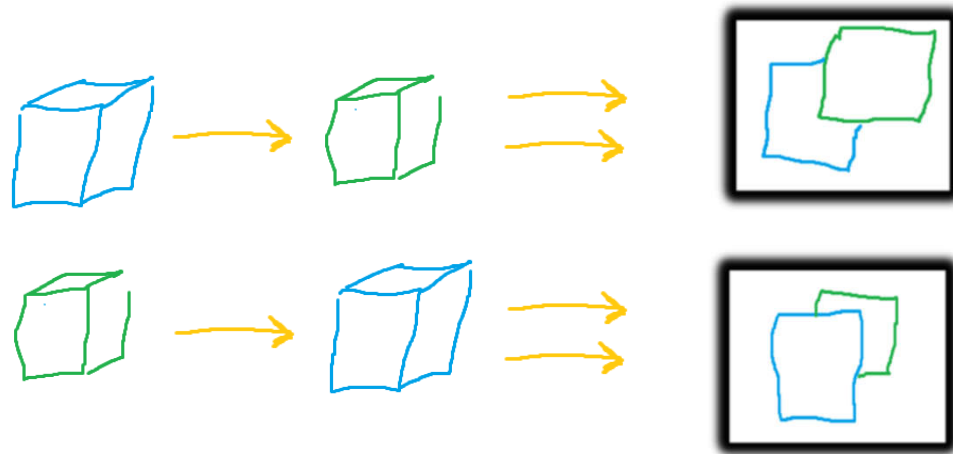
Otras técnicas

- Buffer profundidad.
- Atenuación Luz distancia.
- Mipmapping.
- Skybox.
- Niebla.
- Alpha.



Depth buffer.

Double buffer, z-buffer o buffer de profundidad



Depth buffer.

- El z-buffer de profundidad es un búffer que, almacena información por fragmento, tiene (generalmente) el mismo ancho y alto que el búffer de color y una profundidad (float) de 16, 24 o 32 bits.
- Su función es comparar el valor de cada fragmento con el anterior y en función del resultado se sustituirlo en el color-buffer.
- Este proceso se realiza después del fragment shader.
- Este buffer tiene las mismas coordenadas que el buffer de color FragCoord y corresponde con el viewport del mismo por lo tanto sus coordenadas se sitúan en el 0,0 del mismo izq, abajo.
- Hoy en día, la mayoría de las GPU admiten una característica de hardware llamada prueba de profundidad temprana que hace que se realice el test antes de del fragment shader lo que incrementa la performance del sistema. Siempre que esté claro que un fragmento no será visible (está detrás de otros objetos), podemos descartarlo prematuramente.

```
glEnable( GL_DEPTH_TEST);  
glClear (GL_DEPTH_BUFFER_BIT);
```

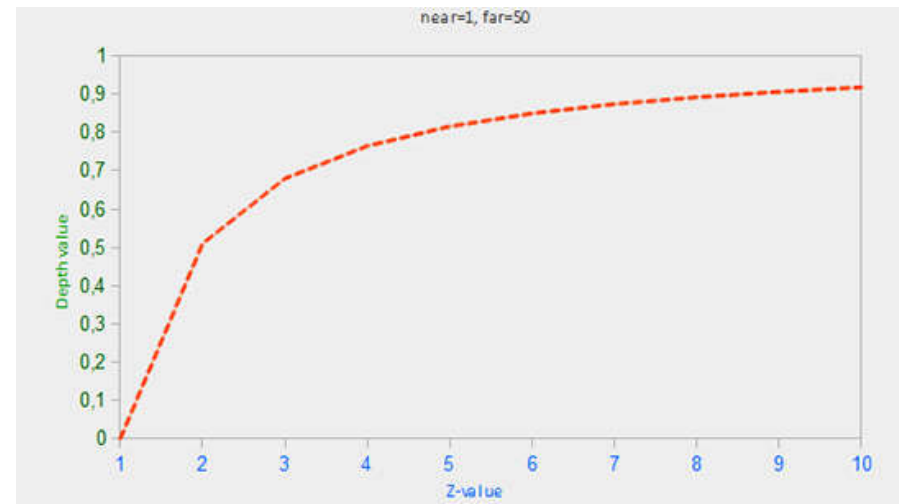
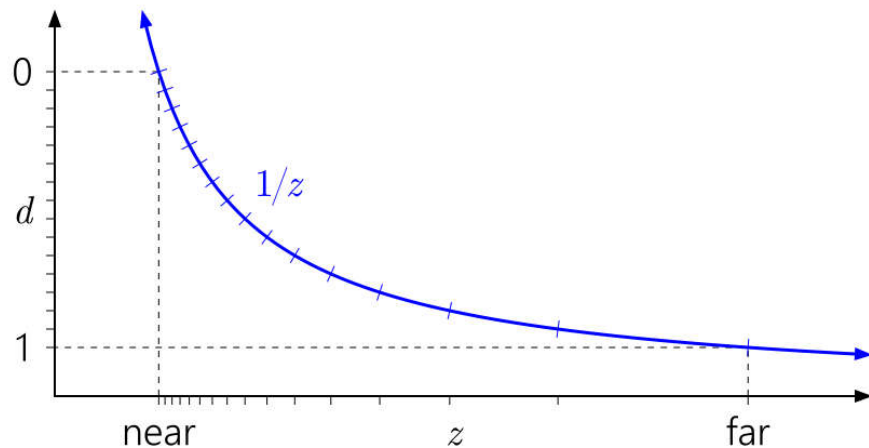
Depth buffer.

- Una vez habilitado, OpenGL realiza los cálculos automáticamente de este modo se descartan los fragmentos si fallaron, no pasan el test de profundidad.
 - » glEnable(GL_DEPTH_TEST);
 - » glClear (GL_DEPTH_BUFFER_BIT);
- Es posible deshabilitar el buffer de profundidad temporalmente, de esta forma el fragmento pasará y se sobre escribirá en el buffer de color.
- OpenGL nos permite modificar los operadores de comparación que utiliza para la prueba de profundidad. Esto nos permite controlar cuándo OpenGL debe pasar o descartar fragmentos y cuándo actualizar el búfer de profundidad. Podemos establecer el operador de comparación (o función de profundidad) llamando a glDepthFunc,
GL_NEVER, **GL_LESS**, GL_EQUAL, GL_LEQUAL, GL_GREATER, GL_NOTEQUAL, GL_GEQUAL, and GL_ALWAYS
- Por ejemplo glDepthFunc(GL_ALWAYS), haría que los fragmentos fueran siempre escritos implementando por tanto el algoritmo del pintor, los fragmentos serán escritos según sean enviados. O podríamos hacer que el buffer funcionara al revés dibujando los objetos ocultos, frente a los que están al revés.

Depth buffer.

- El búfer de profundidad contiene valores de profundidad entre 0.0 y 1.0. Far es el valor lejano y near es el cercano. Su valor no es lineal ya que en general la precisión en los valores cercanos debe ser más alta que en los valores lejanos. La ecuación que lo representa suele tener dos la forma. $1/z$ o

$$F_{depth} = \frac{1/z - 1/near}{1/far - 1/near}$$



Depth buffer 3.3.

Es posible visualizar el z-buffer representado en el fragment shader por `gl_FragCoord.z` con podemos transformar facilmente su valor 0-1, es escala de grises.

```
void main() {  
    FragColor = vec4(vec3(gl_FragCoord.z), 1.0);  
}
```

Los valores así obtenidos son no lineales podemos linearizarlos del siguiente modo para valores .1, 10. Usando como partida la ecuación antes planteada.

```
#version 330 core  
out vec4 FragColor;  
float near = 0.1;           //Los podríamos pasar al shader  
float far = 10.0;  
float no_lineal(float depth) {  
    float z = depth * 2.0 - 1.0;  
    return (2.0 * near * far) / (far + near - z * (far - near)); }  
void main() {  
    float depth = lineal(gl_FragCoord.z) / far;  
    FragColor = vec4(vec3(depth), 1.0);  
}
```

$$F_{depth} = \frac{1/z - 1/near}{1/far - 1/near}$$

Ejercicio Depth buffer.

Utiliza los fragment shader planteado en la transparencia anterior. En la práctica S.S. y grúa de las Transformaciones y ve como cambia el sombreado de los objetos en función de la distancia.

```
void main() {  
    FragColor = vec4(vec3(gl_FragCoord.z), 1.0);  
}
```

```
#version 330 core  
out vec4 FragColor;  
float near = 0.1;           //Los podríamos pasar al shader  
float far = 100.0;  
float no_lineal(float depth) {  
    float z = depth * 2.0 - 1.0;  
    return (2.0 * near * far) / (far + near - z * (far - near)); }  
void main() {  
    float depth = linear(gl_FragCoord.z) / far;  
    FragColor = vec4(vec3(depth), 1.0);  
}
```

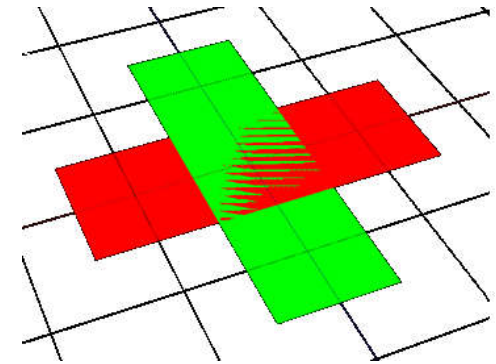
Problemas con el Depth buffer.

Z-fighting

Un artefacto visual común puede ocurrir cuando dos planos o triángulos están tan cercanos que el búfer de profundidad no tiene suficiente precisión para determinar cuál de las dos formas está frente a la otra. El resultado es que las dos formas parecen cambiar continuamente de orden, lo que causa patrones extraños los objetos están “peleando” por quién se ve.

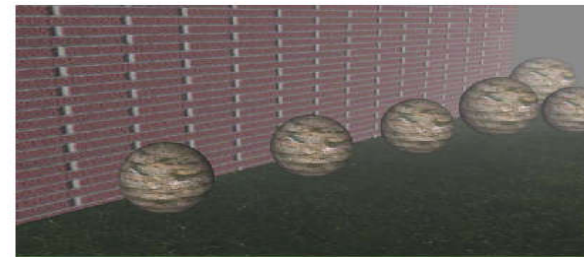
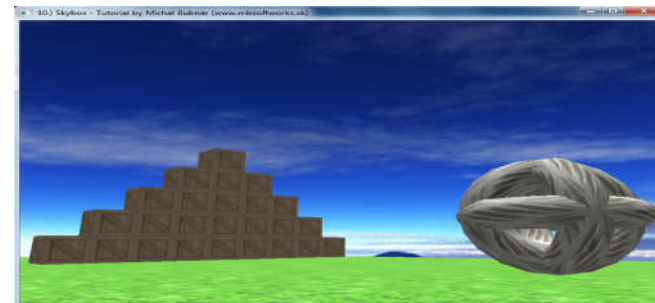
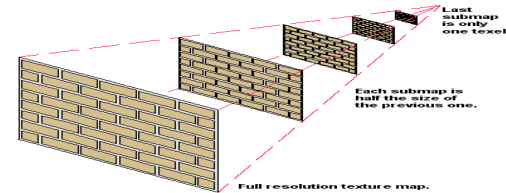
Corrección

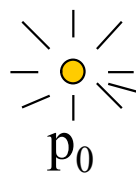
- Aumentar la precisión z-buffer.(no es siempre posible).
- Elegir bien near y Far.
- Separar un poco los objetos.



Otras técnicas

- Buffer profundidad.
- Atenuación Luz distancia.
- Mipmapping.
- Skybox.
- Niebla.
- Alpha.





Atenuación Luz con la distancia

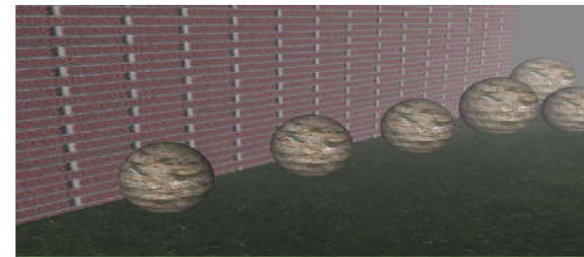
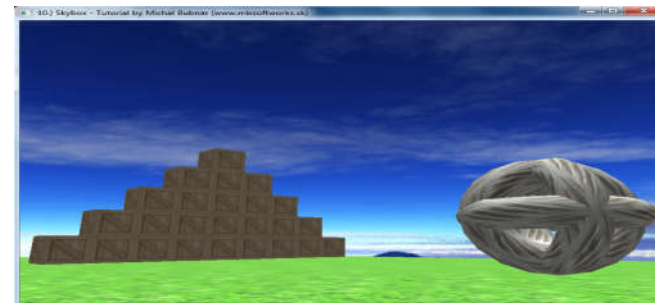
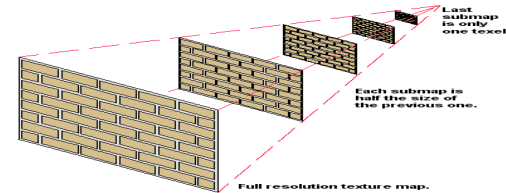


$$I = \frac{1}{a + bd + cd^2} k_a L_a + \left(k_d L_d \mathbf{l} \cdot \mathbf{n} + k_s L_s (\mathbf{r} \cdot \mathbf{v})^\alpha \right) +$$

- La intensidad de iluminación de las luces decrece a medida que aumenta la distancia. Salvo para las luces situadas en el infinito
- En OpenGL:
 - **a: GL_CONSTANT_ATTENUATION**
 - **b: GL_LINEAR_ATTENUATION**
 - **c: GL_QUADRATIC_ATTENUATION.**
- OpenGL establece: $a = 1, b = 0, c = 0$.
- Modificación de los valores:
 - **void glLightf(GL_LIGHTx, GL_CONSTANT_ATTENUATION, value);**
- Característica de emisión no modificada:
 - Atenuación solo influye sobre las componentes de ambiente y difusa de las luces.
- El uso de atenuación influye negativamente en la eficiencia del cálculo

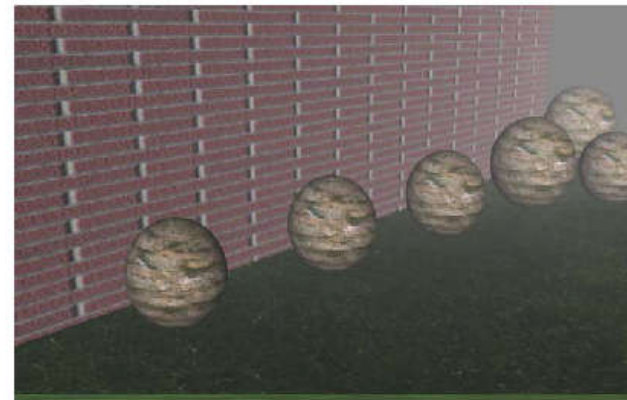
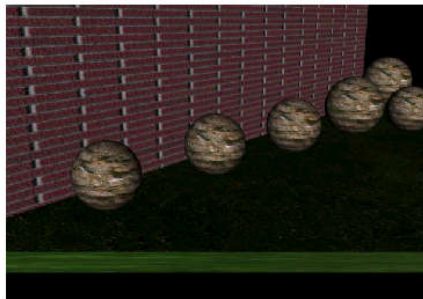
Otras técnicas

- Buffer profundidad.
- Atenuación Luz distancia.
- Mipmapping.
- Skybox.
- Niebla.
- Alpha.



Niebla

- Efecto especial utilizado para difuminar una escena dando la impresión de que se halla realmente cubierta por una masa de niebla
- En general consiste en difuminar el color de la escena, oscureciendo los objetos más lejanos llegando incluso a desaparecer, mientras que los objetos más cercanos aparecen más claros
- El efecto niebla se puede aplicar de dos formas principalmente:
 - Basado en tablas (calculo).
 - Basado en vértices



Aplicación de la Niebla

- El efecto de niebla en OpenGL es muy sencillo, basta con activar la niebla y especificar la densidad, color y tipo de niebla.

- La activación se realiza con:
—glEnable (GL_FOG)

$$f = end - z / end - start$$

$$f = e^{- (density \cdot z)}$$

$$f = e^{- (density \cdot z)^2}$$

- La densidad, color y tipo con:

—void **glFogfv**(GLenum pname, const GLfloat *params)

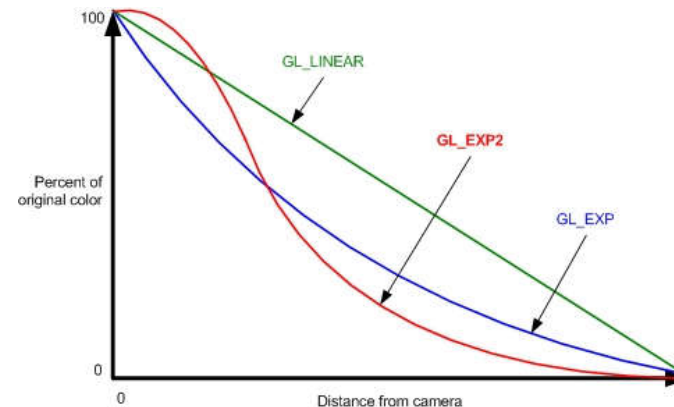
- **glFogfv**, GL_FOG_DENSITY, GL_FOG_START, GL_FOG_END, GL_FOG_INDEX, and GL_FOG_COLOR.

- **glFogi** (GL_FOG_MODE,

- GL_LINEAR

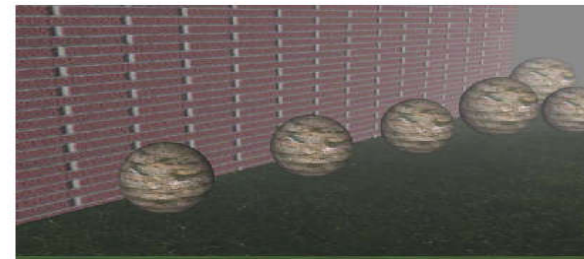
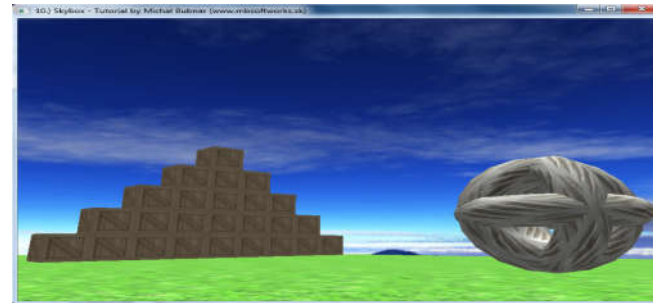
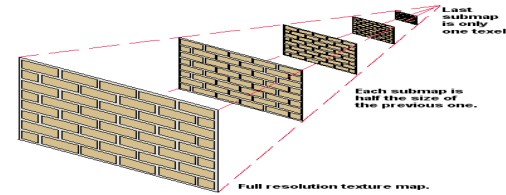
- GL_EXP

- GL_EXP2

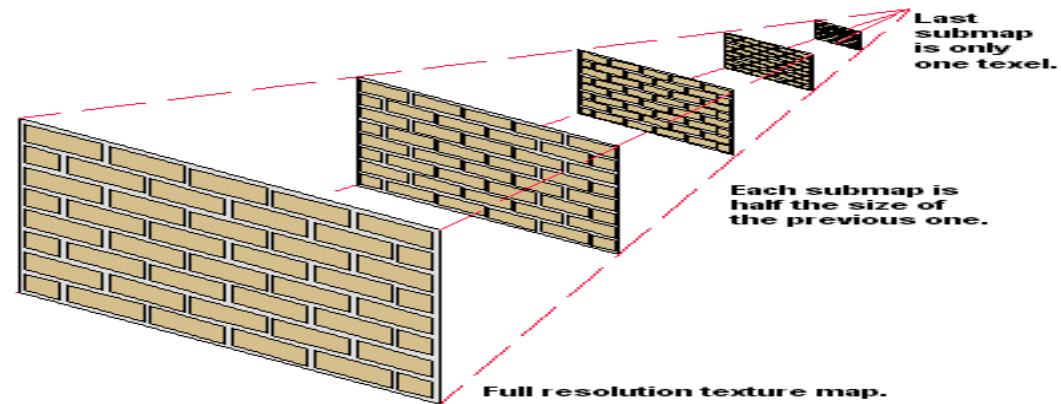


Otras técnicas

- Buffer profundidad.
- Atenuación Luz distancia.
- Mipmapping.
- Skybox.
- Niebla.
- Alpha.



Mipmapping



```
gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, ancho, alto, GL_RGBA, GL_UNSIGNED_BYTE ,ptr);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

`gluBuild2DMipmaps` — builds a two-dimensional mipmap

`GLint gluBuild2DMipmaps(GLenum target, GLint internalFormat, GLsizei width, GLsizei height, GLenum format, GLenum type, const void * data);`

Mipmapping

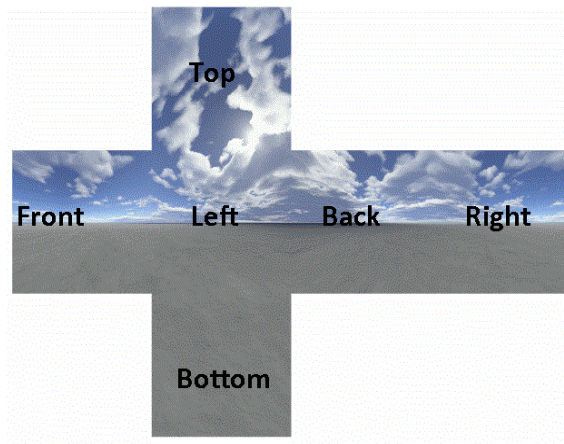
Deberemos modificar la función de lectura de la textura para que se le haga.

```
unsigned char *data = stbi_load(path, &width, &height, &nrComponents, 0);
    if (data)
    {

        gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, width, height, GL_RGB, GL_UNSIGNED_BYTE, data); //con
        mimap
        glGenerateMipmap(GL_TEXTURE_2D);
    }
```

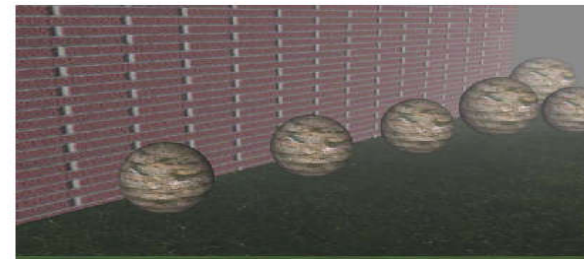
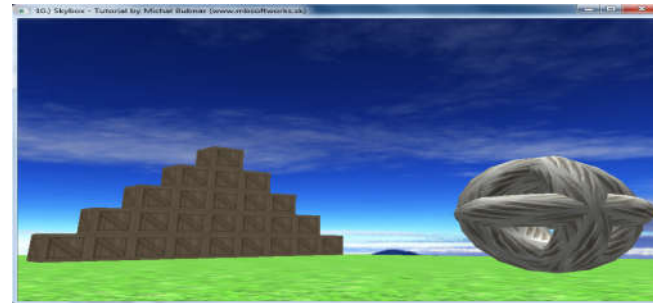
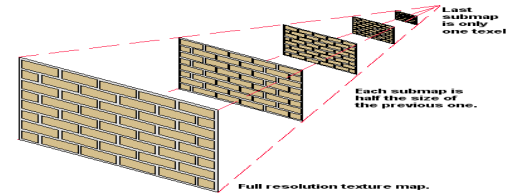
Skybox

Se trata de una caja que contiene el escenario y que permite dar la impresión de cielo. Imagen de <http://www.mbsoftworks.sk/>.
Tambien puede utilizarse un Dome media circunferencia



Otras técnicas

- Buffer profundidad.
- Atenuación Luz distancia.
- Mipmapping.
- Skybox.
- Niebla.
- Alpha.



Colisiones



Que vamos a estudiar

- Introducción
 - Bounding-box
- Choque esfera-esfera
- Choque caja-caja
- Arboles, mapas..

Bounding-box

- Para el cálculo de las colisiones es necesario determinar si se intersecan cada una de las caras de un objeto con las de otro.
- Para simplificarlo, en una primera aproximación se recurre a las esferas, cajas, cilindros, elipsoides que envuelven el objeto.

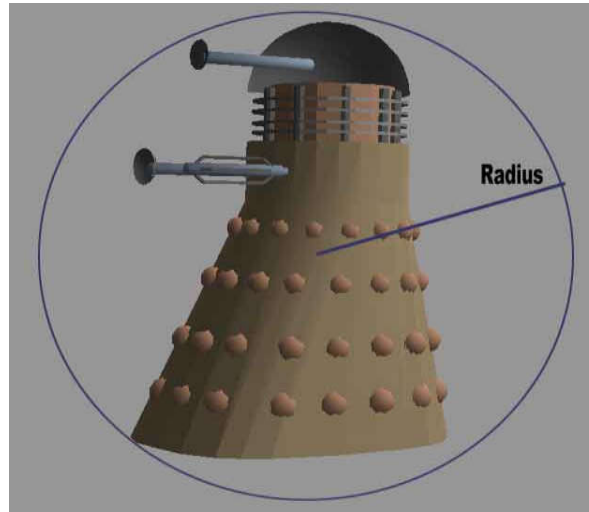


Bounding-esferas

Una esfera engloba el objeto.

Se caracteriza por centro, radio:

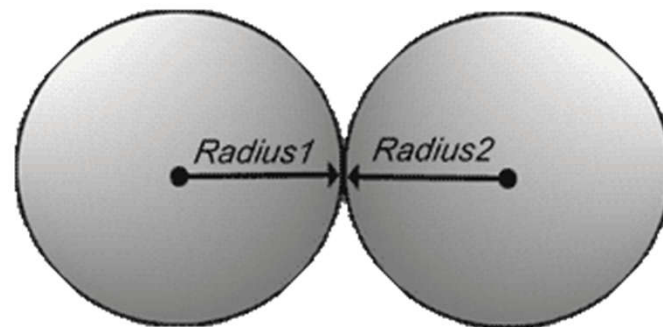
- Centro: punto, float x,y,z
- Radio: float



Esfera-Esfera

- Simple La distancia entre sus centro es menor que la suma de sus radios.

```
float distaciadospuntos(punto a, punto b) {  
  
    return (sqrt((a.x-b.x) * (a.x-b.x) + (a.y-b.y) * (a.y-b.y) + (a.z-b.z) * (a.z-b.z)));  
}  
  
int esferaesfera (esfera a, esfera b) {  
  
    if (distaciadospuntos(a.centro,b.centro) <= a.radio+b.radio)  
    {  
        printf ("tocado\n"); return 1;  
    }  
    else return 0;  
}
```

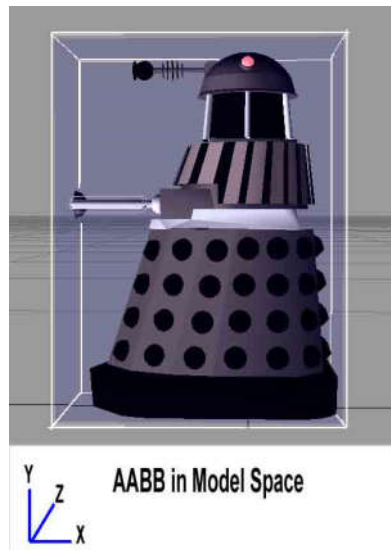
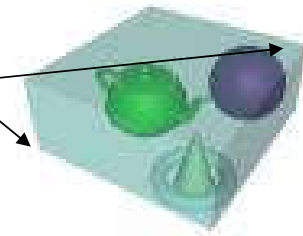


Bounding-box

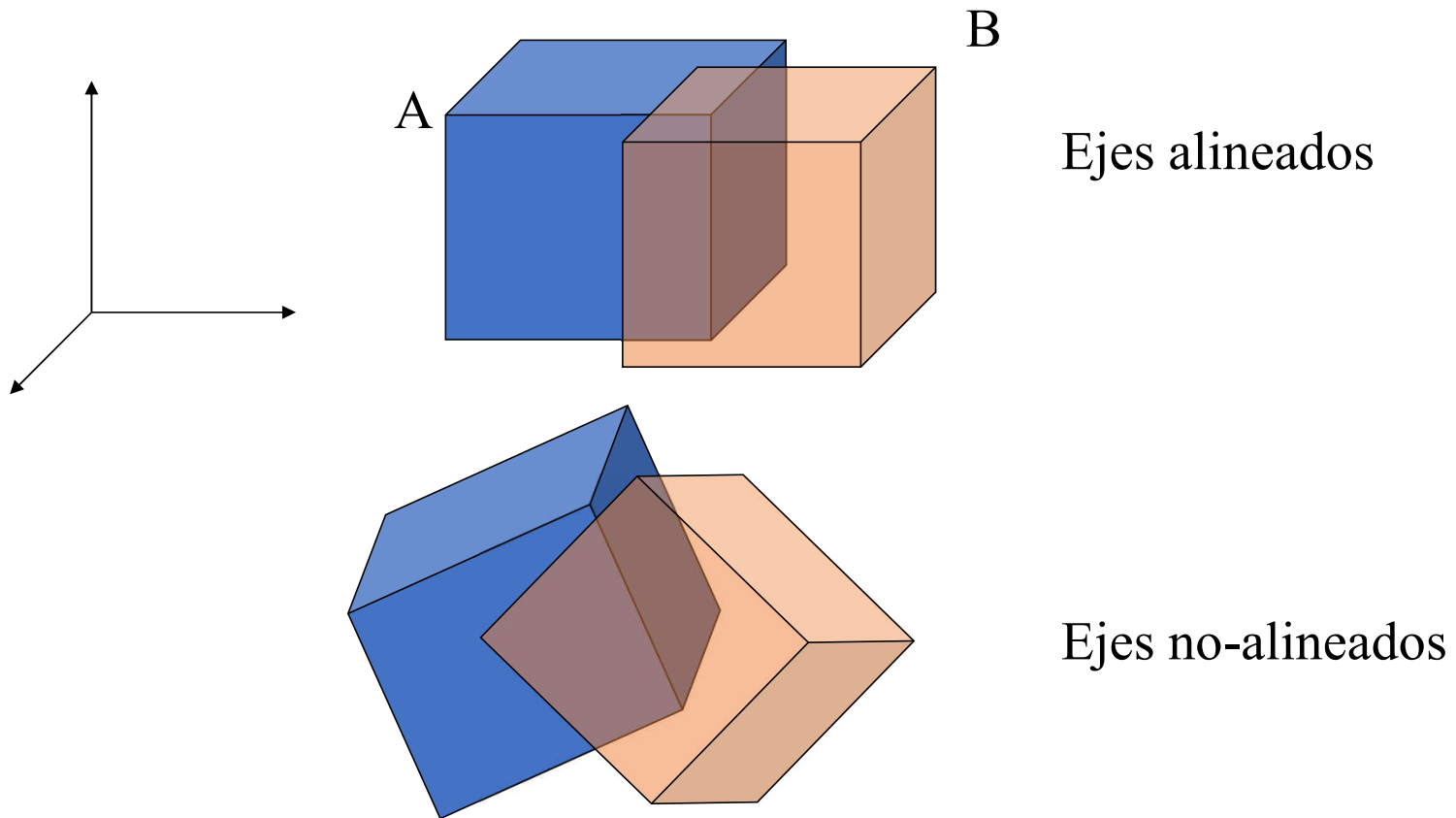
Una caja engloba el objeto, se caracteriza por dos puntos
minBounds and maxBounds :

minBounds : punto, float x,y,z;

maxBounds : punto, float x,y,z

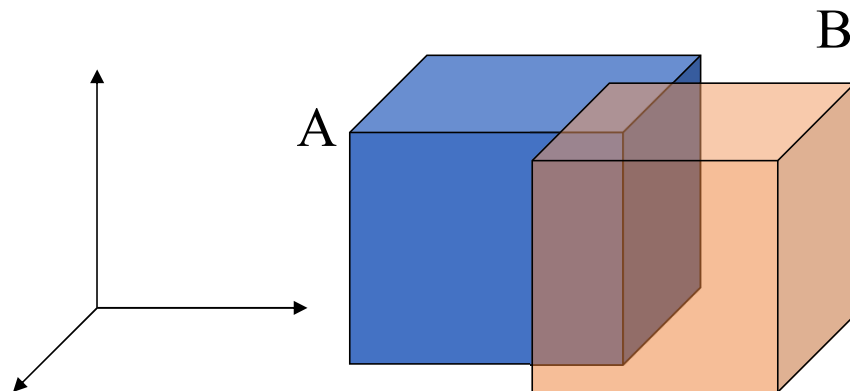


Hay dos posibilidades



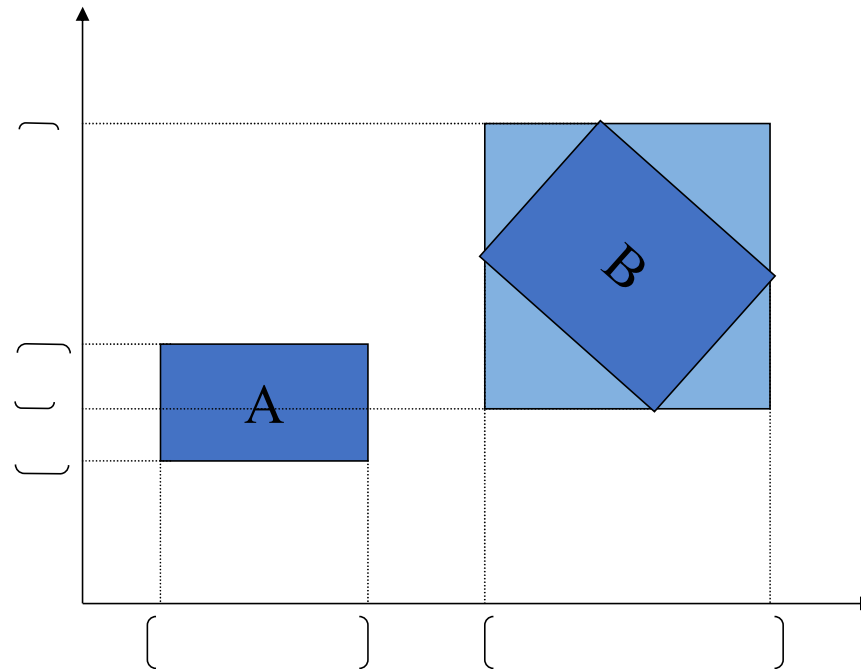
Axis Aligned Bounding Boxes (AABB)

- El uso de cajas alineadas es el procedimiento más utilizado ya que es fácil, rápido y barato de calcular. Lo que hacemos es determinar si las esquinas cada de sus ocho esquinas está dentro de la caja del otro.
 - $A_{xMin} < B_{xMax}$ and $A_{xMax} > B_{xMin}$
 - AND
 - $A_{yMin} < B_{yMax}$ and $A_{yMax} > B_{yMin}$
 - AND
 - $A_{zMin} < B_{zMax}$ and $A_{zMax} > B_{zMin}$



Test Ejes Separados. SAT

- Es un método simple aplicable a segmentos, triángulos y cajas no alineadas
- Se basa en:
 - Dos objetos A y B son disjuntos si para un vector de proyección $\mathbf{v}$ $\mathbf{v} \cdot \mathbf{A}$ y $\mathbf{v} \cdot \mathbf{B}$ son disjuntos.



Árboles, particionado del espacio Mapas

- Árboles.
- Particionado del espacio
- Mapas

